

Ecole Supérieure de Technologie ESSAOUIRA

Filière : Informatique Décisionnelle et Science de Données

Département : Informatique

Rapport de projet de fin d'études

La langue des Signes par l'Apprentissage Automatique

Réalisé par :

Aziza AMAKASSOU

Malak BAHRI

Encadré par :

Mr. Said BENKIRANE

Année universitaire : 2023/2024

Résumé

Le but de ce projet était d'utiliser le machine learning pour classifier quelle lettre de l'alphabet de la Langue des Signes Américaine est signée, étant donné une image d'une main en train de signer au temps réel. Ce projet constitue une première étape vers la construction d'un traducteur potentiel de langue des signes, capable de prendre des communications en langue des signes et de les traduire en langage écrit et oral. Un tel traducteur abaisserait considérablement la barrière pour de nombreuses personnes sourdes et muettes afin de mieux communiquer avec les autres dans les interactions quotidiennes.

Ce but est en outre motivé par l'isolement ressenti au sein de la communauté sourde. La solitude et la dépression sont plus fréquentes chez la population sourde, surtout lorsqu'elle est immergée dans un monde entendant. De grandes barrières qui affectent profondément la qualité de vie découlent du décalage de communication entre les sourds et les entendants. Quelques exemples en sont la privation d'informations, la limitation des connections sociales et la difficulté d'intégration dans la société.

La plupart des implémentations de recherche pour cette tâche ont utilisé des cartes de profondeur générées par une caméra de profondeur et des images haute résolution. L'objectif de ce projet était de voir si les réseaux neuronaux sont capables de classifier les lettres signées en utilisant des images simples de mains prises avec un appareil personnel tel qu'une webcam d'ordinateur portable. Cela est en accord avec la motivation car cela rendrait une future implémentation d'un traducteur en temps réel de la langue des signes vers le langage oral/écrit pratique dans une situation quotidienne.

Remerciements

Nous souhaitons exprimer notre profonde gratitude avant d'entamer la rédaction de ce rapport à travers ces courtes lignes :

Nous sommes reconnaissants envers l'École Supérieure de Technologie d'Essaouira pour son engagement à fournir une éducation de qualité et à promouvoir l'importance des Projets de Fin d'Études. Nous sommes honorés d'avoir eu l'opportunité de vivre cette expérience enrichissante et sommes convaincus que cela aura un impact positif sur notre formation future.

Nous tenons à exprimer notre reconnaissance envers les personnes qui nous ont apporté leur soutien, que ce soit d'ordre technique ou moral, dans l'élaboration de ce travail.

Nous souhaitons présenter notre profond respect et reconnaissance au Professeur Said BENKIRANE, notre encadrant de projet de fin d'études, pour nous avoir permis de travailler sur un sujet aussi passionnant, pour son enseignement et le temps qu'il nous a consacré. Chacune de nos réunions avec lui était un réel plaisir, nous encourageant à approfondir nos connaissances en Machine Learning.

Nous ne saurions conclure cette partie sans penser à nos familles et amis, dont le soutien omniprésent nous permet d'aller de l'avant et de donner le meilleur de nous-mêmes.

Table des matières

I.	Introduction	7
II.	La Structure Globale du projet	8
A.	Traitement des Données	8
B.	Entraînement.....	8
1.	Teachable Machine	8
2.	Les caractéristiques principales de Teachable Machine :	9
C.	Classification des Gestes	9
III.	La source des données.....	10
A.	ASL.....	10
1.	Langue Naturelle.....	10
2.	Communication Gestuelle.....	10
3.	Grammaire Visuelle	10
4.	Diversité Régionale.....	11
5.	Évolution et Adaptation	11
6.	Reconnaissance et Accessibilité.....	11
B.	Collection de données :.....	11
1.	Le signe yes.....	11
C.	Traitement des données	11
1.	Détection des Mains.....	12
2.	Extraction de la Région d'Intérêt (ROI)	12
3.	Redimensionnement de l'Image	12
4.	Prétraitement de l'Image	12
IV.	Le modèle Machine Learning	12
A.	L'apprentissage par transfert de connaissance	13
1.	Fine tuning	13
2.	Les étapes du processus de Fine-Tuning.....	13
B.	Dépendances Logicielles et Modules Importés	18
1.	Modules Importés	18
2.	IDE : Environnement de Développement Intégré.....	19
3.	Framework	19
4.	HTML et CSS	20
V.	Défis rencontrés.....	21
VI.	Réalisation.....	21
A.	Intégration.....	21

B.	L'interface	21
C.	La prédiction.....	22
VII.	Les perspectives d'avenir.....	23
VIII.	Conclusion	24
IX.	Bibliographie.....	25

Table des figures

Figure 1: diagramme de blocs du projet.....	8
Figure 2 : la langue des signes Américaine.....	10
Figure 3 : la collection des signes	11
Figure 4 :: la collection des lettres	11
Figure 5 : l'application Teachable machine	14
Figure 6 : teachable machine_ choix du projet	14
Figure 7: teachable machine définition des classes.....	15
Figure 8 : teachable machine importation des images	15
Figure 9 : teachable machine _ train the model	16
Figure 10 : teachable machine_ validation du modèle.....	17
Figure 11 : teachable machine _téléchargement du modèle.....	17
Figure 12 : logo de opencv.....	18
Figure 13 : logo de Os	18
Figure 14 : logo de time	18
Figure 15 ; logo de numpy	19
Figure 16 : logo de Tensorflow	19
Figure 17 : logo de CvZone.....	19
Figure 18 : logo de Pycharm	19
Figure 19 : logo de Flask.....	19
Figure 20 : logo de HTML et CSS	20
Figure 21 : interface sign language detection.....	21
Figure 22 : detection du signe Hello	22
Figure 23 : concerver l'image 'hello sign'	23

I. Introduction

La communication est un pilier fondamental de la société, permettant aux individus d'exprimer leurs besoins, leurs émotions et leurs idées. Cependant, pour les personnes sourdes ou malentendantes, la communication peut être un défi majeur, en particulier dans un monde dominé par la langue parlée. La langue des signes offre une alternative essentielle, permettant à ces individus de communiquer efficacement avec le monde qui les entoure.

Dans le cadre de ce projet, notre objectif était de construire un modèle capable de classifier quel mot de la Langue des Signes Américaine (ASL) est signée, à partir d'une image d'une main en train de signer. Ce projet constitue une première étape vers la création d'un éventuel traducteur de langage des signes, capable de prendre des communications en langage des signes et de les traduire en langage écrit et oral. Un tel traducteur réduirait considérablement la barrière pour de nombreuses personnes sourdes ou muettes de mieux communiquer avec les autres dans leurs interactions quotidiennes.

Cette démarche est motivée par l'isolement ressenti au sein de la communauté sourde. La solitude et la dépression sont plus répandues parmi la population sourde, surtout lorsqu'ils sont immergés dans un monde entendant. De larges barrières qui affectent profondément la qualité de vie découlent de la déconnexion de communication entre les sourds et les entendants. Quelques exemples en sont la privation d'informations, la limitation des connexions sociales et la difficulté d'intégration dans la société.

La plupart des implémentations de recherche pour cette tâche ont utilisé des cartes de profondeur générées par une caméra de profondeur et des images haute résolution. L'objectif de ce projet était de voir si les modèles d'Apprentissage automatique sont capables de classifier les mots signés de l'ASL en utilisant des images simples de mains prises avec un appareil personnel tel qu'une webcam d'ordinateur portable. Cela est en phase avec la motivation, car cela rendrait une implémentation future d'un traducteur de langage des signes en temps réel en langage oral/écrit pratique dans une situation quotidienne.

Dans ce rapport, nous détaillerons les différentes étapes de notre projet, de la collecte et du prétraitement des données à la conception et à l'entraînement de notre modèle de détection de la langue des signes. Nous discuterons également des défis rencontrés, des techniques utilisées et des résultats obtenus, ainsi que des perspectives d'avenir pour cette technologie prometteuse. Enfin, nous présenterons une évaluation critique de notre système, mettant en lumière ses forces et ses limitations, ainsi que des recommandations pour des améliorations futures.

II. La Structure Globale du projet

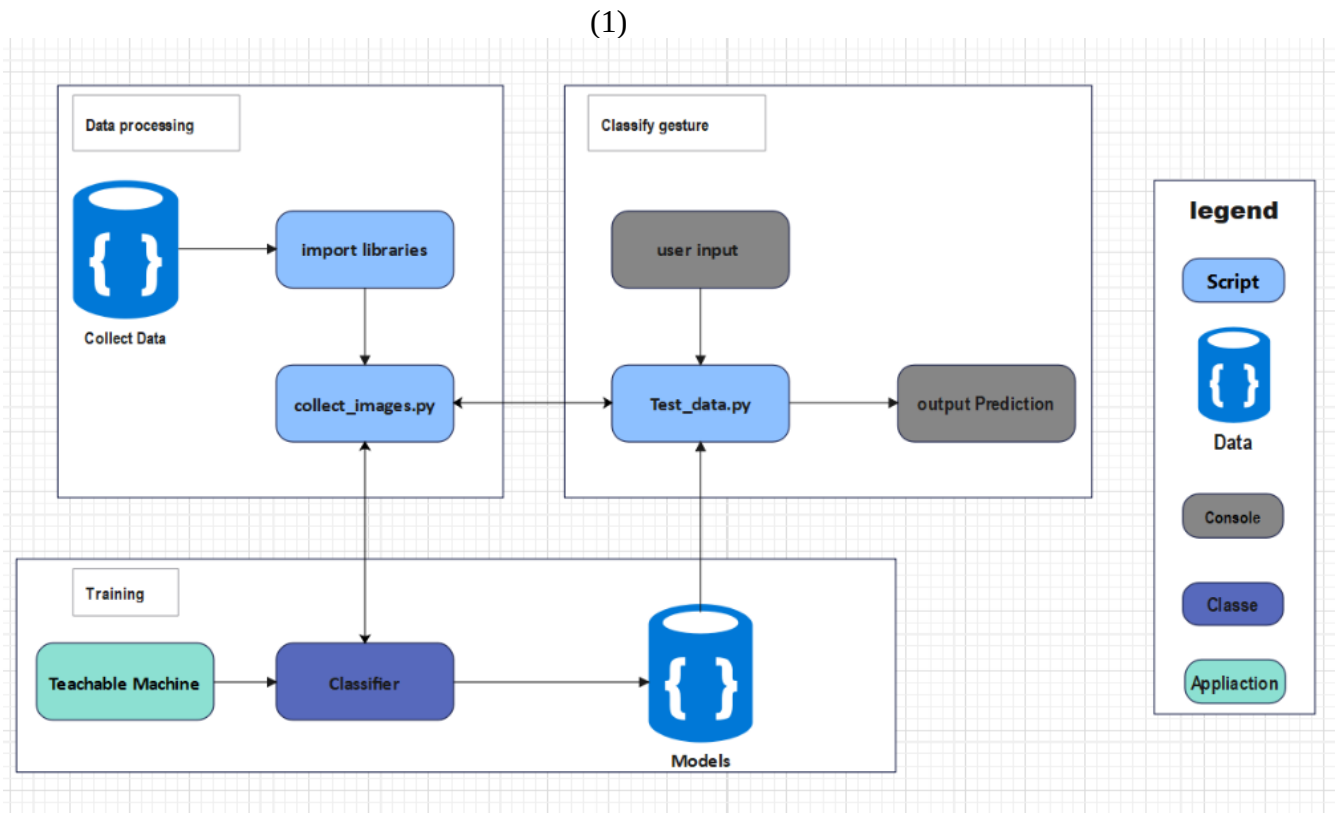


Figure 1: diagramme de blocs du projet

Comme illustré dans la **Figure 1**, le projet sera structuré en 3 blocs fonctionnels distincts : **Traitement des Données**, **Entraînement** et **Classification** des Gestes. Le diagramme de blocs est simplifié pour abstraire certains détails :

A. Traitement des Données

Le script **collect_images.py** collecte les images à partir d'une webcam et les enregistrer dans le stockage de fichiers. Les images sont prétraitées pour redimensionner, filtrer et améliorer les caractéristiques. Pendant l'entraînement, les données d'images sont divisées en ensembles de données d'entraînement, de validation et de test.

B. Entraînement

1. Teachable Machine

Teachable machine est une application en ligne développée par Google qui permet aux utilisateurs de créer leurs propres modèles d'apprentissage automatique sans nécessiter de connaissances préalables en programmation ou en science des données. L'application est conçue pour être conviviale et accessible à tous, des débutants aux experts en apprentissage automatique.

2. Les caractéristiques principales de Teachable Machine :

a) Interface Conviviale :

L'interface de Teachable Machine est intuitive et conviviale, ce qui permet aux utilisateurs de créer, d'entraîner et de tester leurs propres modèles d'apprentissage automatique sans rencontrer de difficultés techniques.

b) Apprentissage par le Faire

Teachable Machine adopte une approche d'apprentissage par le faire, où les utilisateurs peuvent expérimenter directement avec leurs propres données et voir instantanément les résultats de leur modèle.

c) Entraînement Simple

Les utilisateurs peuvent facilement télécharger leurs propres données (images, sons ou poses) et les étiqueter en quelques clics. L'application prend ensuite en charge le processus d'entraînement du modèle en utilisant ces données étiquetées.

d) Personnalisation

Teachable Machine permet aux utilisateurs de personnaliser différents aspects de leur modèle, tels que le choix de l'algorithme (classification, régression ou segmentation) et des paramètres spécifiques à chaque algorithme.

e) Visualisation des Résultats

Une fois le modèle entraîné, les utilisateurs peuvent visualiser les performances de leur modèle en temps réel à l'aide de différents outils de visualisation fournis par l'application.

f) Exportation Facile

Une fois satisfait des performances de leur modèle, les utilisateurs peuvent facilement l'exporter dans différents formats compatibles avec diverses plateformes de développement.

C. Classification des Gestes

Une fois le modèle entraîné, il peut être utilisé pour classer de nouveaux gestes ASL. Le script `test_data.py` charge une image individuelle, puis utilise le modèle pour **prédire** le geste.

III. La source de données

A. ASL

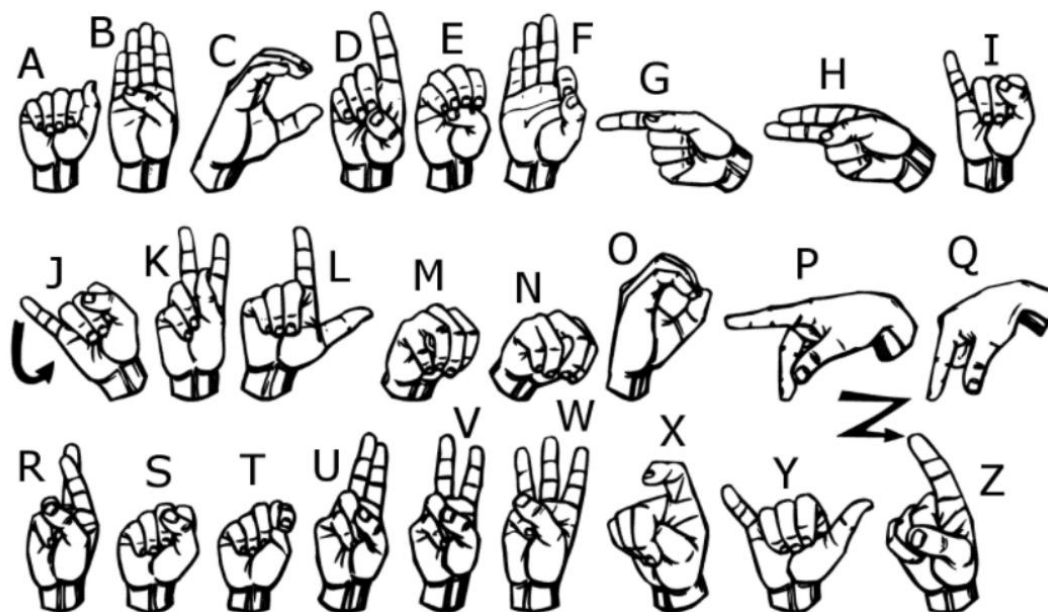


Figure 2 : la langue des signes Américaine

ASL, ou American Sign Language, est une langue visuelle et gestuelle utilisée principalement par les communautés sourdes et malentendantes aux États-Unis et au Canada. Contrairement à la langue des signes britannique (BSL) ou à d'autres langues des signes, ASL est une langue à part entière avec sa propre grammaire, syntaxe et vocabulaire distincts. (2)

Quelques points clés à connaître sur ASL :

1. Langue Naturelle

ASL est une langue naturelle utilisée par les personnes sourdes aux États-Unis et dans certaines régions du Canada. Elle a évolué de manière organique au fil du temps au sein de la communauté sourde, avec des règles grammaticales et linguistiques établies.

2. Communication Gestuelle

ASL repose sur un système de communication gestuelle où les signes, réalisés avec les mains, les expressions faciales et le langage corporel, sont utilisés pour communiquer des idées, des concepts et des sentiments.

3. Grammaire Visuelle

Contrairement aux langues verbales, ASL utilise une grammaire visuelle qui se base sur des aspects tels que la direction, la configuration des mains, le mouvement et les expressions faciales pour transmettre des informations.

4. Diversité Régionale

Tout comme les langues parlées, ASL peut présenter des variations régionales. Les signes et les expressions peuvent différer d'une région à une autre, même au sein des États-Unis et du Canada.

5. Évolution et Adaptation

Comme toute langue vivante, ASL continue d'évoluer et de s'adapter aux besoins et aux contextes changeants de la communauté sourde. De nouveaux signes peuvent être créés pour refléter les avancées technologiques, les concepts modernes et les changements sociétaux.

6. Reconnaissance et Accessibilité

ASL gagne en reconnaissance et en acceptation dans la société, avec des efforts croissants pour rendre les informations et les services accessibles aux personnes sourdes en utilisant cette langue.

B. Collection de données :

Nous avons collecté les données à partir de la caméra de l'ordinateur. Le script **collect_images.py**. A permis d'utiliser la fonction **cv2.VideoCapture(0)** pour capturer les images en temps réel à partir de la webcam de l'ordinateur. En itérant sur chaque classe prédéfinie, nous avons utilisé la fonction **cv2.imwrite()** pour enregistrer chaque image capturée dans le stockage de fichiers avec un nom de fichier unique. Pour chaque classe de signe telles que **"Hello"**, **"I love You"**, **"Yes"**, etc., nous avons pu capturer et enregistrer **300 images** différentes, qui ont été ensuite prétraitées et utilisées dans l'entraînement du modèle.

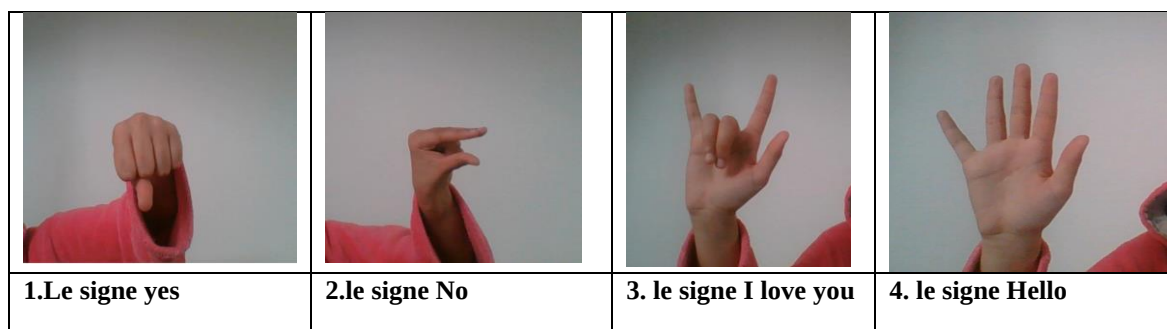


Figure 3 : la collection des signes

C. Traitement de données

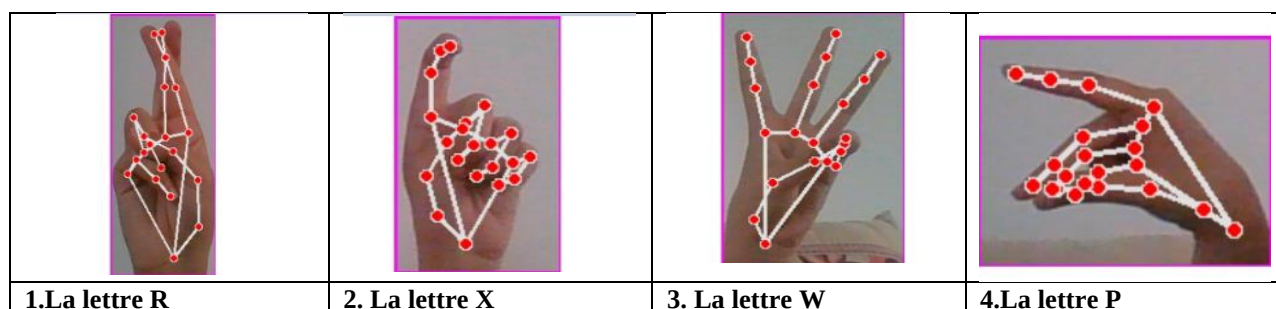


Figure 4 : la collection des lettres

1. Détection des Mains

On a utilisé un modèle de détection de la main pour localiser et suivre les mains dans le flux vidéo en temps réel. Cela est réalisé en utilisant la bibliothèque **cvzone.HandTrackingModule**.

2. Extraction de la Région d'Intérêt (ROI)

Une fois qu'une main est détectée, le script extrait une région d'intérêt (ROI) autour de la main en utilisant les coordonnées du rectangle englobant (bbox) fournies par le modèle de détection de main.

3. Redimensionnement de l'Image

La région d'intérêt est redimensionnée pour correspondre à une taille spécifique. Cela garantit que toutes les images traitées ont la même taille, ce qui est important pour l'entraînement et la classification cohérente du modèle.

4. Prétraitement de l'Image

Avant de passer l'image à travers le modèle de classification, on a effectué un certain prétraitement. En créant une image blanche et place la région d'intérêt redimensionnée au centre de cette image.

IV. Le modèle Machine Learning

En utilisant un processus de fine-tuning ou l'apprentissage par transfert de connaissance, l'application **Teachable Machine** a joué un rôle essentiel dans le processus de développement de notre projet en facilitant la collecte des données, l'entraînement du modèle, la validation des performances et l'intégration du modèle entraîné dans le script pour une utilisation pratique en temps réel.

A. L'apprentissage par transfert de connaissance

1. Fine tuning

a) Définition

Il s'agit d'un **processus itératif qui vise à améliorer la performance du modèle sur une tâche particulière**, sans perdre les connaissances préalables acquises lors de l'entraînement initial.

L'idée centrale réside dans la capacité du modèle à généraliser à de nouveaux domaines tout en conservant sa capacité à se spécialiser. Le Fine-Tuning permet d'**ajuster les poids des connexions entre les neurones** pour qu'ils soient plus adaptés à la nouvelle tâche sans perturber significativement les connaissances préexistantes.

Un modèle pré-entraîné peut être fine-tuné pour la **détection d'objets spécifiques** dans un contexte particulier comme les véhicules autonomes ou les caméras de surveillance. (3)

2. Les étapes du processus de Fine-Tuning

Avant de débiter le Fine-Tuning, **on évalue les performances initiales du modèle sélectionné sur la tâche cible**. Cela fournit une base de référence qui permettra de mesurer l'amélioration par la suite.

Le processus commence par :

a) La collecte et la préparation des données

Nous avons rassemblé les données en utilisant la caméra de l'ordinateur, puis enregistrer chaque image capturée dans le stockage de fichiers.

b) Le nettoyage des données

Cela est également indispensable pour éliminer les erreurs, les doublons et les incohérences. Ce formatage facilite ensuite l'affinage.

Les images sont traitées par le modèle, redimensionnées implicitement à la taille attendue par le modèle et implémenter des filtres pour extraire les caractéristiques pertinentes des images et faciliter la détection des signes.

c) Choix du modèle pré-entraîné en fonction de la tâche spécifique

(4)

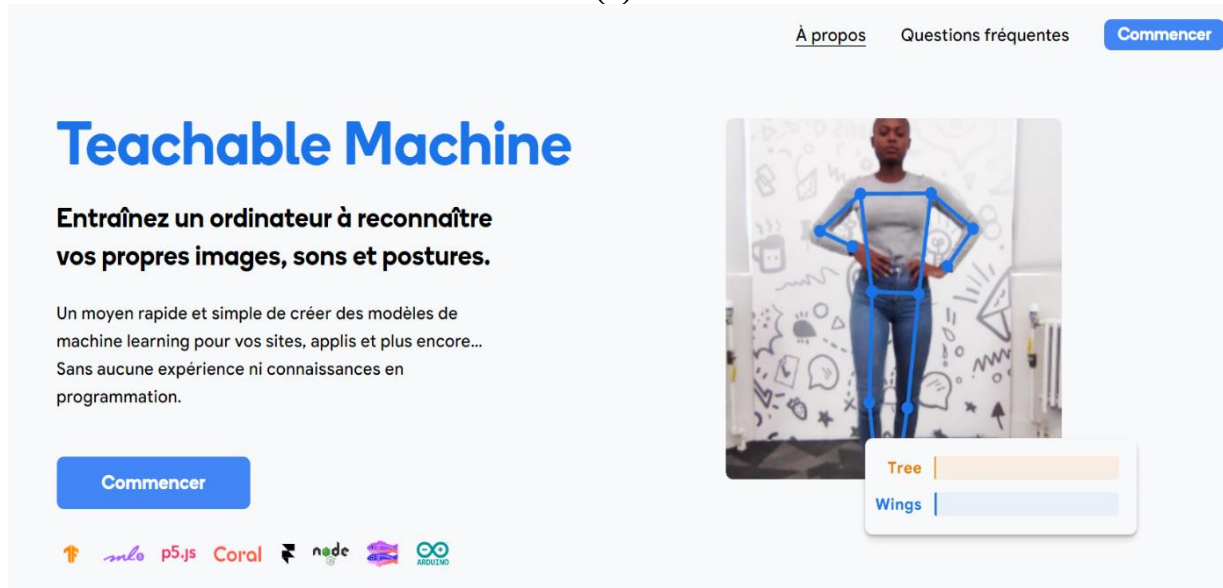


Figure 5 : l'application Teachable machine

Teachable Machine contient des modèles pré-entraînés qui peuvent servir de point de départ pour créer des modèles personnalisés. Ces modèles pré-entraînés sont disponibles pour différents types de tâches d'apprentissage automatique, tels que la classification d'images, la détection d'objets, la segmentation sémantique, etc.

Lorsque vous utilisez Teachable Machine, vous avez la possibilité de choisir parmi ces modèles pré-entraînés en fonction de la tâche que vous souhaitez accomplir.

(1) Sélection du Projet

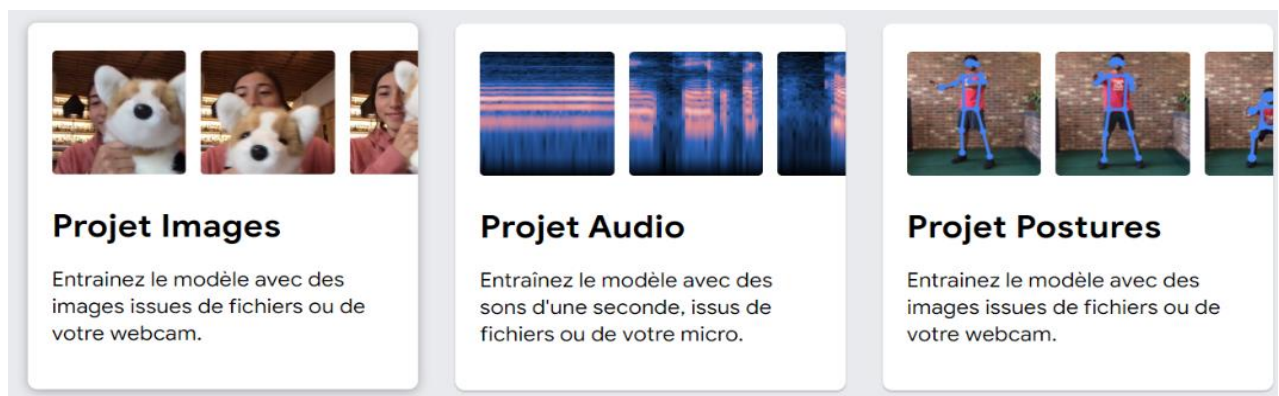


Figure 6 : teachable machine_ choix du projet

Lors du démarrage dans **Teachable Machine**, nous avons choisi de créer un projet "Images" afin de classer des images de mains effectuant différents signes de la langue des signes.

(2) Définition des Classes

Ensuite, nous avons défini les différentes classes ou catégories que nous voulions inclure dans notre modèle. Chaque classe correspondait à un signe de la langue des signes que nous souhaitions détecter, comme "**Hello**", "**I love you**", "**Yes**", les **lettres**.

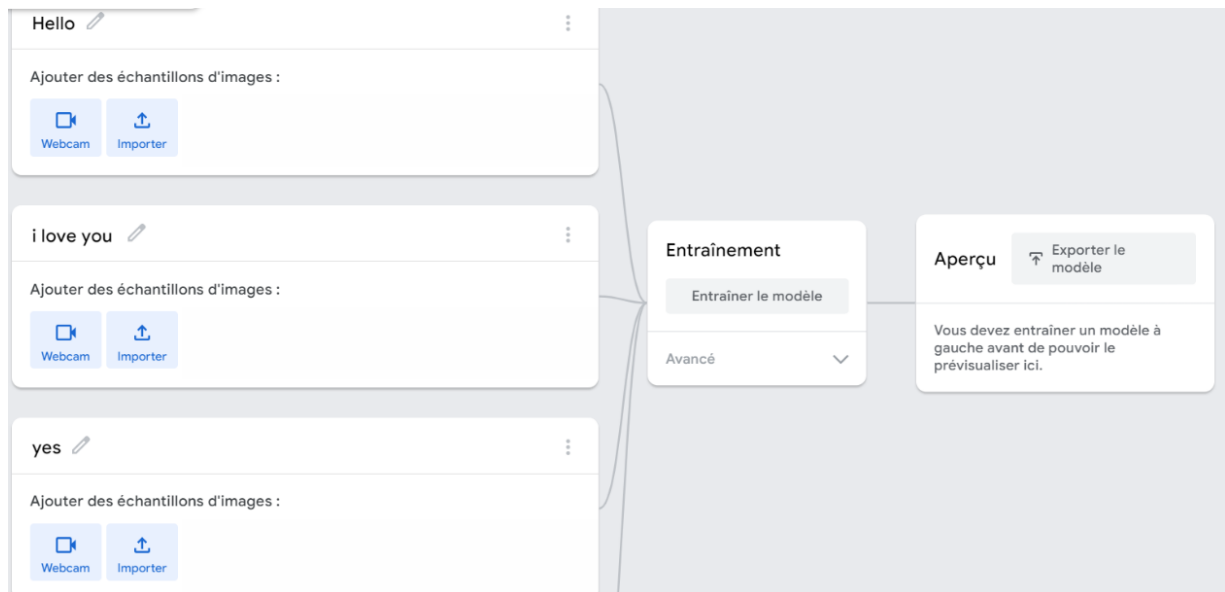


Figure 7: teachable machine définition des classes

(3) Importation des Images

Pour chaque classe définie, nous avons importé des images correspondantes montrant des mains effectuant le signe associé. Il était important d'avoir une diversité d'exemples pour chaque classe afin que notre modèle puisse apprendre à reconnaître efficacement ces signes dans diverses situations.

Exemple de la classe R :

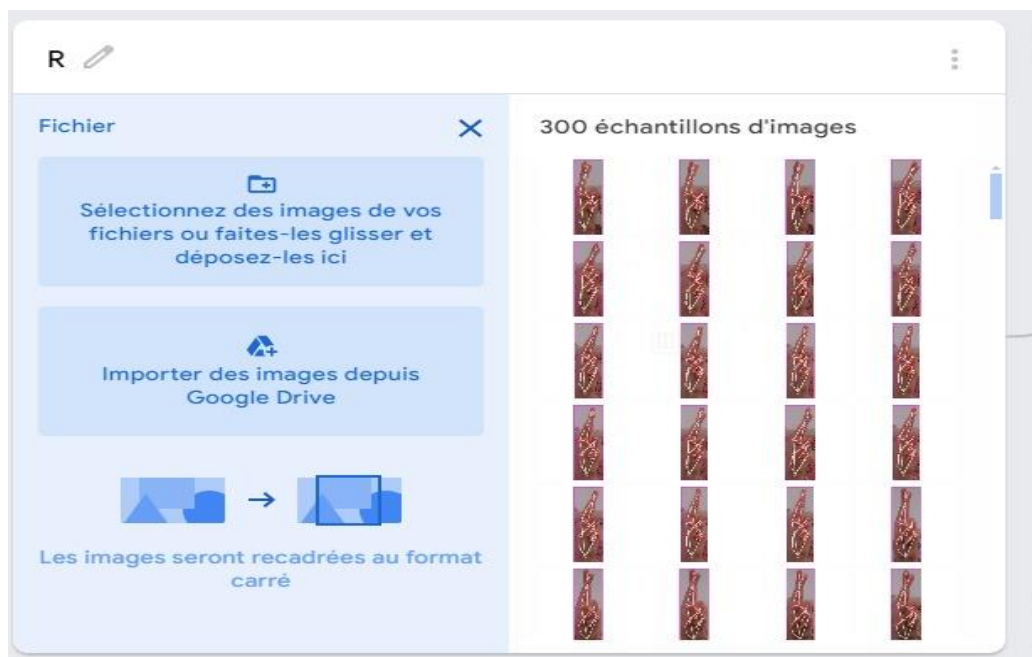


Figure 8 : teachable machine importation des images

(4) Entraînement du Modèle

Une fois que toutes les images ont été importées et que les classes ont été définies, nous avons initié le processus d'entraînement en cliquant sur le bouton **Train**. Teachable Machine a alors utilisé les images fournies pour ajuster les paramètres du modèle et apprendre à distinguer correctement les différentes classes.



Figure 9 : teachable machine _ train the model

(5) Évaluation du Modèle

Après l'entraînement, nous avons évalué les performances du modèle pour nous assurer qu'il fonctionnait comme prévu. Cela impliquait souvent de tester le modèle sur un ensemble de données de validation distinct pour vérifier sa capacité à généraliser au-delà des données d'entraînement.

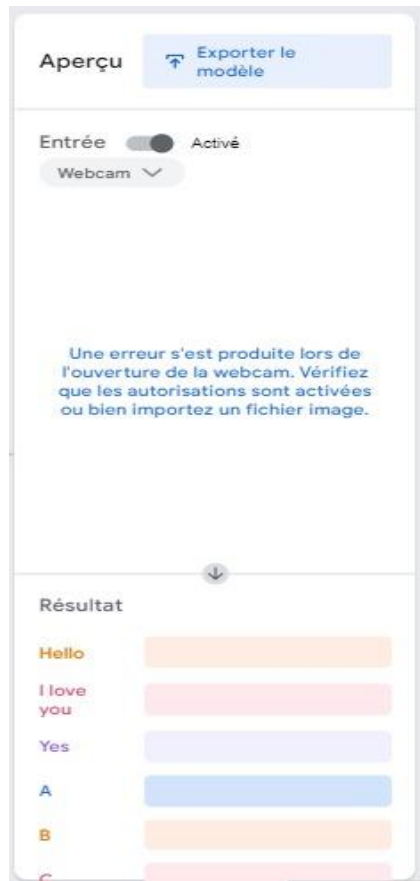


Figure 10 : teachable machine_validation du modèle

(6) Téléchargement du Modèle

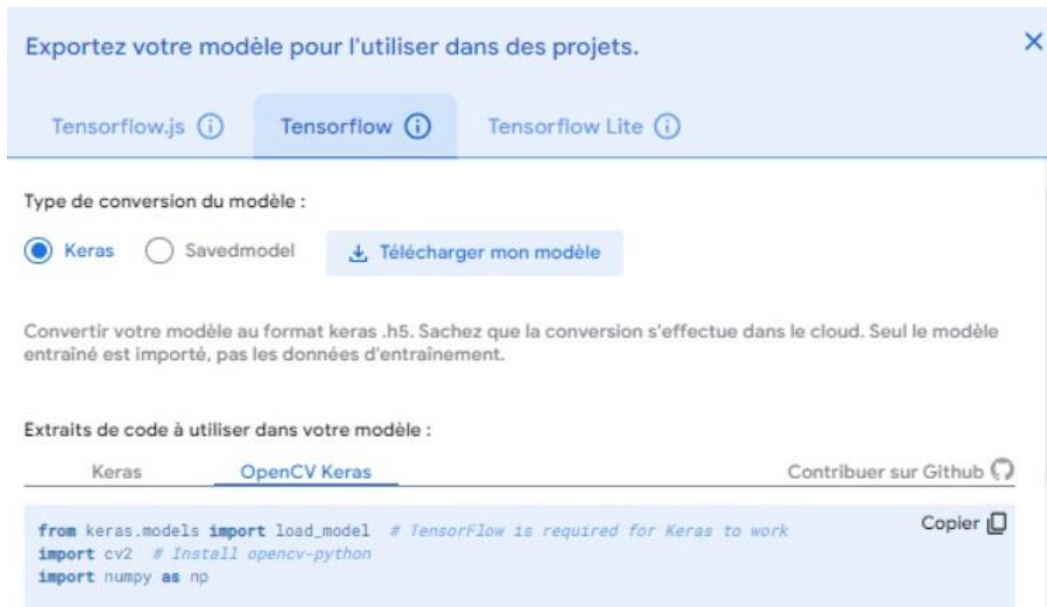


Figure 11 : teachable machine_téléchargement du modèle

Une fois satisfait des performances du modèle, nous l'avons téléchargé à partir de Teachable Machine. Le modèle entraîné était généralement téléchargé dans un format compatible avec les différentes plates-formes de développement, nous permettant de l'utiliser dans notre environnement de programmation, nous l'avons téléchargé sous format **.h5** avec un fichier **.txt** nommé **labels** qui contient les **id** de chaque classe.

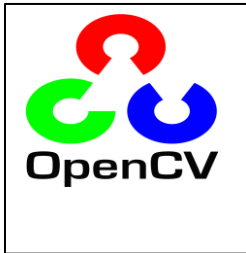
B. Dépendances Logicielles et Modules Importés

1. Modules Importés

Nous avons utilisé plusieurs bibliothèques logicielles et modules Python pour faciliter le développement de nos scripts.

Les principales bibliothèques utilisées :

a) OpenCv



Dans notre projet, la bibliothèque OpenCV (cv2) a joué un rôle crucial dans la capture et le traitement des images capturées. Nous avons pu accéder à la webcam de l'ordinateur et capturer des images en temps réel pour notre ensemble de données.

Figure 12 : logo de opencv

b) OS



Le module **OS** en Python est un outil puissant qui permet d'interagir avec le système d'exploitation sous-jacent. Il offre une variété de fonctionnalités pour la gestion des fichiers et des répertoires, ainsi que pour l'exécution de commandes système.

Figure 13 : logo de Os

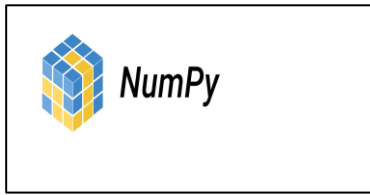
c) Time



Le module **Time** en Python est un outil essentiel pour la gestion du temps et des délais dans vos programmes.

Figure 14 : logo de time

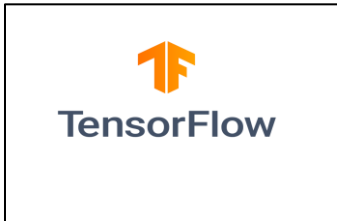
d) Numpy



La bibliothèque NumPy est une composante essentielle de l'écosystème Python pour le traitement des données numériques.

Figure 15 ; logo de numpy

e) Tensorflow



La bibliothèque TensorFlow utilisée pour la construction et l'entraînement de modèles de détection d'objets.

Figure 16 : logo de Tensorflow

f) cvzone



La bibliothèque CVZone, fournit divers modules pour le traitement d'images et le suivi de la main. Dans notre cas, la fonction HandDetector est utilisée pour détecter les mains dans le flux vidéo.

Figure 17 : logo de CvZone

2. IDE : Environnement de Développement Intégré



Figure 18 : logo de Pycharm

PyCharm est un environnement de développement intégré (IDE) développé par JetBrains, conçu spécialement pour les développeurs Python. Il nous a fourni un environnement de développement productif et efficace pour créer notre application de détection des signes de la langue des signes en Python.

3. Framework Flask



Figure 19 : logo de Flask

Flask est un framework web léger pour Python, conçu pour faciliter le développement d'applications web de manière simple et extensible.

Flask a été un choix judicieux pour notre projet d'application de détection des signes de la langue des signes, nous offrant un moyen simple et flexible de créer une API web pour notre application. Son approche minimaliste et son extensibilité nous ont permis de rapidement mettre en place notre application.

a) Création d'une API Web

Flask nous a permis de créer une API web pour notre interface de détection des signes de la langue des signes. Nous avons pu définir des routes URL et associer des fonctions de traitement pour gérer les requêtes HTTP entrantes et renvoyer des réponses appropriées.

b) Rendu de Pages Web

Flask offre également des fonctionnalités pour générer du contenu HTML dynamique. Nous avons utilisé des templates **Jinja2** pour rendre des pages web dynamiques en utilisant les données générées par notre interface.

c) Gestion des Requêtes et des Réponses

Flask simplifie la gestion des requêtes et des réponses HTTP. Nous avons pu accéder aux données des requêtes entrantes, valider les paramètres, et renvoyer des réponses HTML.

d) Intégration avec d'autres Bibliothèques

Flask peut être facilement intégré avec d'autres bibliothèques Python, telles que **OpenCV** pour le traitement d'images, ou des bibliothèques d'apprentissage automatique comme **TensorFlow** pour l'analyse des données.

e) Déploiement Simple

Flask permet un déploiement simple d'applications web sur diverses plates-formes, que ce soit localement pour le développement ou sur des serveurs en production.

4. HTML et CSS



Figure 20 : logo de HTML et CSS

HTML (HyperText Markup Language) et CSS (Cascading Style Sheets) jouent un rôle essentiel dans la construction de notre interface de détection des signes de la langue des signes.

- HTML est utilisé pour définir la structure de base de la page web. Dans notre cas, nous utilisons HTML pour créer une disposition claire de la page, avec des éléments tels que les en-têtes, les sections, et les boutons d'interaction.
- CSS est utilisé pour styliser l'apparence de l'interface utilisateur. Nous pouvons spécifier les couleurs, les polices, les tailles de texte, les boutons, et d'autres propriétés visuelles pour chaque élément HTML.

V. Défis rencontrés

Un des principaux défis rencontrés dans ce projet a été la collecte de données pour entraîner le modèle de reconnaissance des signes. Le processus de collecte de données s'est avéré chronophage et exigeant en efforts, en particulier compte tenu de la nécessité de capturer un large éventail de gestes pour une reconnaissance précise. Le fait de ne pas avoir accès à une base de données existante a rendu cette tâche encore plus ardue, nécessitant la création de notre propre ensemble de données à partir de zéro. Cette démarche a impliqué de prendre des photos de nous-mêmes effectuant différents gestes, ce qui a demandé du temps et de la patience. De plus, le fait de ne pas avoir inclus toutes les lettres de l'alphabet, comme J, K, et Z, dans notre ensemble de données initial représente un défi supplémentaire, car cela limite la capacité du modèle à reconnaître un ensemble complet de gestes.

VI. Réalisation

A. Intégration

Avec Flask, nous avons intégré une interface dans une application web Python, gérant les requêtes et les réponses HTTP. Pour permettre l'interaction utilisateur, nous avons ajouté des boutons pour prendre et supprimer des photos, et en utilisant JavaScript, nous avons défini des fonctions pour gérer les événements de clic sur ces boutons, utilisant l'API Fetch pour communiquer avec le serveur Flask. Nous avons utilisé la bibliothèque OpenCV pour capturer le flux vidéo de la webcam et l'afficher sur la page web.

B. L'interface

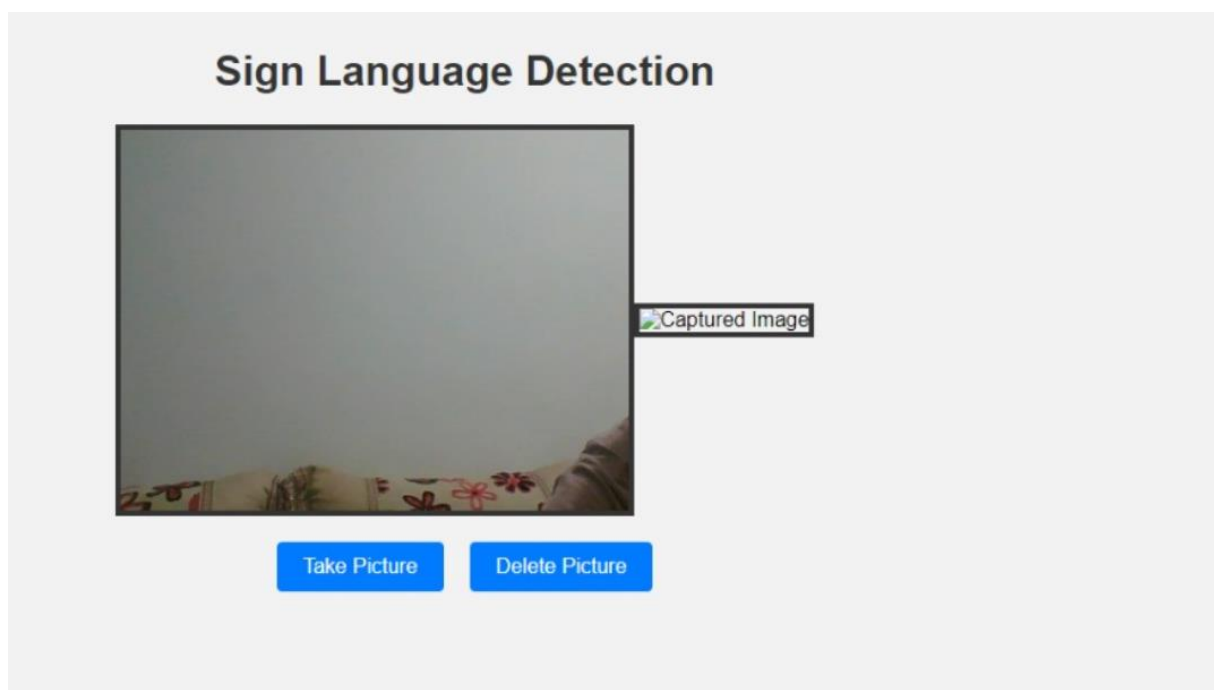


Figure 21 : interface sign language detection

Lors du lancement de notre interface, la caméra s'affiche, prête à capturer les signes effectués par la main pour effectuer la prédiction. L'utilisateur est accueilli par un flux vidéo en temps réel de sa webcam, lui permettant de visualiser ses gestes en direct. À partir de là, deux options s'offrent à lui : prendre une photo du geste actuel en appuyant sur le bouton **Take Picture** ou bien supprimer la photo capturée précédemment en cliquant sur **Delete Picture**. Cette fonctionnalité offre à l'utilisateur une expérience interactive et intuitive, lui permettant de contrôler le processus de capture et de gestion des images pour la prédiction des signes de la langue des signes.

C. La prédiction

(5)

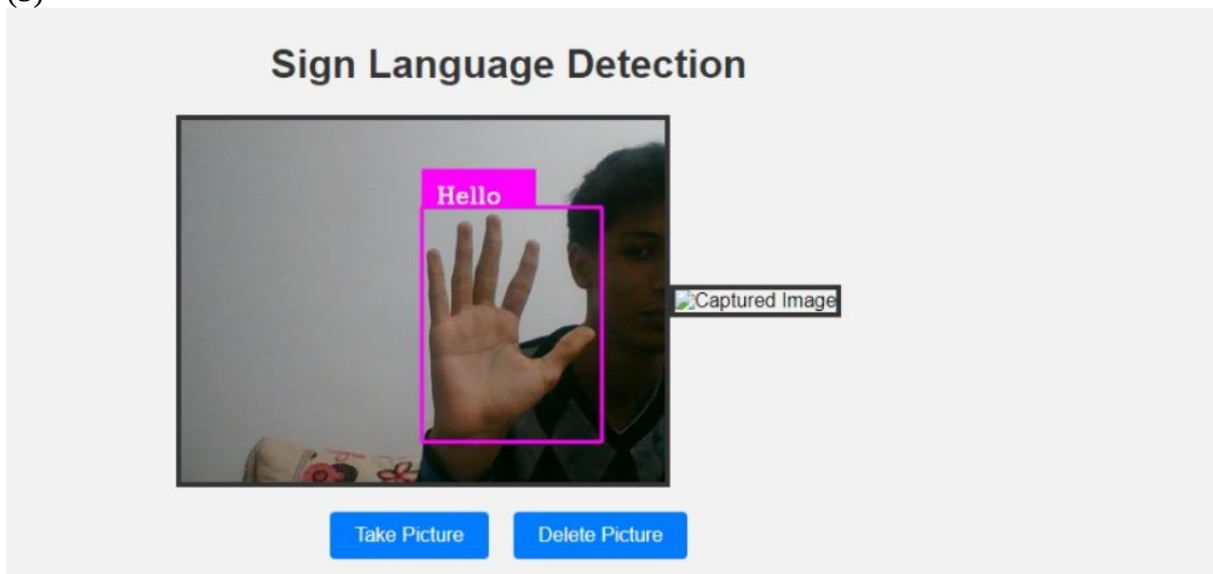


Figure 22 : detection du signe Hello

Une fois que l'utilisateur capture une image en appuyant sur le bouton **Take Picture**, cette image est automatiquement enregistrée dans un dossier nommé **static** de notre application. Ce dossier est spécialement dédié au stockage des ressources statiques telles que les images, les fichiers CSS et JavaScript, entre autres. De même, lorsque l'utilisateur choisit de supprimer une image en appuyant sur **Delete Picture**, le fichier correspondant est immédiatement retiré du dossier **static**, garantissant ainsi une gestion efficace des ressources et évitant l'encombrement inutile de l'espace de stockage. Cette approche assure également la cohérence et la propreté de l'interface, en garantissant que seules les images pertinentes sont conservées et disponibles pour une utilisation ultérieure.

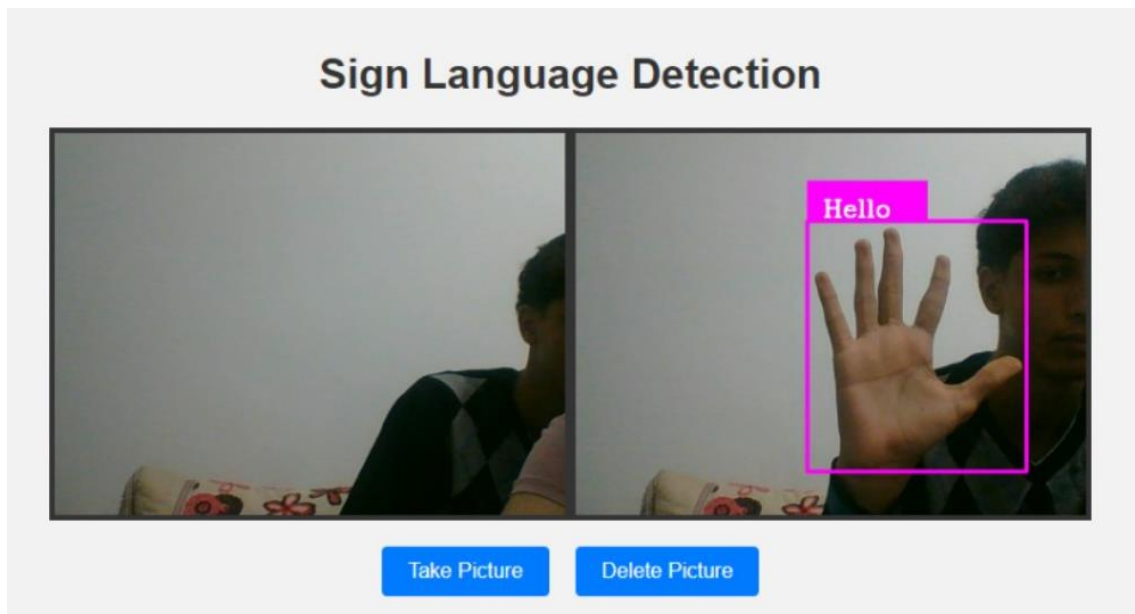


Figure 23 : conserver l'image 'hello sign'

VII. Les perspectives d'avenir

Les perspectives d'avenir de ce projet sont prometteuses, offrant plusieurs voies pour son développement continu. En premier lieu, **l'amélioration de la précision** du modèle reste une priorité, que ce soit par l'ajout de données supplémentaires, le perfectionnement des techniques de prétraitement des images ou l'exploration de modèles de deep learning plus sophistiqués. En parallèle, **l'extension de la base de données** avec une plus grande variété de signes et de locuteurs de la langue des signes peut enrichir la capacité du modèle à généraliser et à reconnaître un éventail plus large de gestes. Une **interface utilisateur** plus conviviale et interactive pourrait également être développée pour améliorer l'expérience globale de l'utilisateur. De plus, **l'intégration de fonctionnalités supplémentaires** telles que **la traduction automatique des signes** en langage écrit ou oral ou la reconnaissance de gestes complexes pourrait étendre les capacités du système. Enfin, des **études approfondies** sur l'accessibilité et l'impact social de cette technologie pourraient fournir des insights précieux pour son optimisation et son déploiement dans divers contextes d'utilisation. En explorant ces pistes, ce projet pourrait contribuer de manière significative à faciliter la communication et l'inclusion des personnes sourdes ou malentendantes dans la société.

VIII. Conclusion

Ce projet de fin d'études représente une contribution significative à la recherche et au développement dans le domaine de la science des données, en mettant en œuvre des techniques de machine learning pour la classification des signes de la Langue des Signes Américaine (ASL) (6) à partir d'images de mains en temps réel. L'objectif de ce projet était de jeter les bases d'un traducteur potentiel de la langue des signes, qui pourrait permettre une communication plus fluide entre les personnes sourdes ou malentendantes et le reste de la société.

En mettant en lumière les défis et les motivations sous-jacents à ce projet, nous avons exploré la pertinence sociale et l'impact potentiel d'une telle technologie. En effet, la communication est essentielle pour le bien-être et l'inclusion sociale, et les personnes sourdes ou malentendantes sont confrontées à des barrières significatives dans ce domaine. Un traducteur de langue des signes en temps réel pourrait contribuer à réduire ces barrières et à favoriser une meilleure communication et compréhension mutuelle.

En décrivant en détail les différentes étapes du projet, de la collecte et du prétraitement des données à la conception et à l'entraînement du modèle de classification, nous avons mis en évidence les aspects techniques et méthodologiques de notre approche. En utilisant des outils tels que Teachable Machine pour l'entraînement du modèle et Flask pour l'intégration de l'interface utilisateur, nous avons démontré la faisabilité et la praticité de notre solution.

Enfin, en évaluant les performances de notre système et en discutant de ses forces et de ses limitations, nous avons ouvert la voie à des améliorations futures et à des recherches complémentaires dans ce domaine.

En conclusion, ce projet représente une contribution importante à la fois sur le plan technique et social, et offre des perspectives prometteuses pour l'avenir du développement de traducteurs de langue des signes basés sur le machine learning (7).

IX. Bibliographie

1. Wondershare EdrawMax.
2. <https://www.britannica.com/topic/American-Sign-Language>. *britannica*. [En ligne]
3. <https://datascientest.com/fine-tuning-tout-savoir>. *datascientest*. [En ligne]
4. <https://teachablemachine.withgoogle.com/>. *teachable machine*. [En ligne]
5. 127.0.0.1 : 5000. *sign lanuaage Detection*. [En ligne]
6. <https://www.nidcd.nih.gov/>. *NIH*. [En ligne]
7. Apprentissage automatique.